

PRODUCT STRATEGY / ENGINEERING PLAN / RELEASE CONTROL

OMNILUME

FINAL CORRECTED MASTER ROADMAP & IMPLEMENTATION BLUEPRINT

Corrected order	18 actions; identity and security foundations precede collaboration; lifecycle precedes realtime; AI is Action 17, the last major feature stage.
Current position	Important room, RLS, lifecycle, Watch, E2EE, Files, Study, Focus Lock, SSR, and deployment foundations exist, but complete product and runtime evidence do not.
Status rule	Source implementation, static validation, local runtime, multi-user verification, database/security verification, and production verification remain distinct.
Release position	PRODUCTION READY: NO until required evidence gates pass.
Safety rule	Extend first; preserve working code; no destructive resets or speculative rewrites.

Protect the existing codebase. Extend first. Verify source, runtime, security, multi-user, and production behavior separately.

Executive Summary

OmniLume is a privacy-conscious collaboration platform combining temporary rooms, synchronized Watch, Study tools, Files, room chat, E2EE private messaging, social profiles, Personal/General Chat, persistent groups, Music, and later AI assistance. The existing codebase is the protected foundation. This corrected blueprint integrates the complete feature vision without treating a component, migration, or button as proof of completion.

Current strengths

RoomRealtimeProvider/useRoomSync is the shared room synchronization anchor.

E2EE private chat uses per-message failure tolerance and an undecryptable state.

Room lifecycle, recovery/permanent objects, cleanup, RLS, storage, Watch v3, Files, Study, Focus Lock, and SSR proxy foundations exist.

Public homepage room controls were removed while authenticated create/join paths remain.

Critical gaps

Google OAuth/profile onboarding and the social graph are incomplete.

Personal/General/group messaging and provider Music are missing.

Current room-to-group conversion conflicts with no-username/group semantics.

Independent runtime, Cron, mobile, performance, and production evidence remain gates.

Current -> Target Product Model

Current foundation	Required evolution	Target state
Rooms, memberships, Watch, Files, Study, room chat, E2EE private chat	Identity, RLS, lifecycle, shared realtime, social graph, groups, Music	Privacy-aware collaboration where Rooms remain separate from Personal Chat and groups are explicit conversions.
Source fixes and partial runtime evidence	Layered verification and release gates	Every feature is evidenced at source/static, runtime, multi-user, database/security, and production layers.

Critical Issues and Release Blockers

Priority	Problem	Impact	Closure condition
P0	Identity/onboarding gap	Google OAuth, profile completion, session behavior, and account policy are required before the complete social product.	Configured Google flow, safe redirect, no duplicates, SSR/client agreement.
P0	Lifecycle was too late in the prior order	Recovery, guests, scheduling, permanent rooms, locks, and cleanup must define authoritative room state before collaboration.	Expiry/recovery/reopen/permanent/cleanup tests pass.
P0	Runtime evidence is incomplete	Source/build success does not prove two-user, Cron, mobile, or production behavior.	Required evidence is recorded; unavailable tests keep readiness NO.
P1	Room-to-group mismatch	Current conversion accepts a group username and calls a permanent-room RPC.	Transactional conversion with no group public username and explicit admin permission.
P1	Master-plan coverage gap	Account switching, reactions/GIFs, metadata/lyrics, study history/music, Lume surfaces, Room Pulse, Memory distinction, and moderation need explicit ownership.	Every item appears in the feature register and action owner.

Dependency Map

Foundation -> release

01 Evidence -> 02 Auth/Identity -> 03 Database/RLS/Storage -> 04 Profiles/Social -> 05 Rooms/Permissions -> 06 Lifecycle/Cleanup -> 07 Shared Realtime -> 08-13 Collaboration/Management -> 14 Conversion -> 15 Personal/General/Groups -> 16 Music -> 17 AI -> 18 Hardening/Production

Why: authentication prevents anonymous feature access; RLS protects data; lifecycle defines availability; realtime synchronizes only authorized state; social/groups depend on identity; Music depends on private storage/providers; AI depends on trustworthy normal records.

Action-by-Action Implementation Roadmap

Each action below is an implementation blueprint. Later actions may integrate earlier capabilities, but they must not create duplicate sources of truth or bypass earlier security/lifecycle boundaries.

01

Foundation and Evidence

P0

PARTIAL

Action purpose / why it exists

Establish one source of truth before adding product surface area.

Current implementation and status

The repository has strong anchors: Next 16 app routes, Supabase clients, RoomRealtimeProvider/useRoomSync, E2EE private chat, room lifecycle, Focus Lock, Watch, Files, Study, proxy, migrations, and canonical plan. Evidence depth is uneven.

What must be fixed or built

Inventory routes/actions/policies/storage/realtime/deployments and map every feature to source, static, local, multi-user, database/security, and production evidence. Create disposable fixtures and a status ledger.

Existing code/features to preserve

Keep current routes, room/chat/Watch/Study/Files foundations, public homepage boundary, and retained legacy files unless reachability proves removal safe.

Dependencies and what it unlocks

Unlocks safe work on every later action.

Database / RLS / storage

No product schema change. Define migration discipline, test-data isolation, secret/logging rules, and rollback/runbook conventions.

Realtime / security

No client-only security gates; no secret/plaintext/ciphertext logging; one authoritative provider per boundary.

UI / UX expected result

Use current navigation and component vocabulary. Every async action has loading, disabled, success, error, empty, retry, accessible, and mobile states. Users see only data/features allowed by identity, membership, lifecycle, and permission.

Risks / regression prevention

- Add the smallest compatible seam and keep the current happy path available.
- Test direct server/database calls as well as UI.
- Use feature flags, additive migrations, and evidence gates; never use hidden UI as security.

Testing and verification

Baseline typecheck/build/lint/diff, route smoke, static searches, database probes, and evidence review. Label every result separately as source/static, local runtime, multi-user/browser, database/security, or production.

Rollback / recovery

Documentation and test fixtures only; never reset data.

Definition of Done

Inventory is current, production data is excluded from destructive tests, and every later action has a gate.

02

Authentication and Identity

P0

PARTIAL

Action purpose / why it exists

Make identity and session state reliable before profiles, groups, providers, or AI.

Current implementation and status

src/proxy.ts refreshes Supabase sessions and protects sensitive routes. Login currently uses email/password; Google OAuth, callback/configuration, profile completion routing, account switching, and recovery policy remain.

What must be fixed or built

Add Google OAuth, callback, loading/error/logout states, safe next redirects, existing-account detection, profile-completion gate, account switching, and documented provider/env configuration. Decide compatibility with existing password accounts.

Existing code/features to preserve

Preserve @supabase/ssr, server/client auth boundaries, protected routes, and existing records. Extend auth; do not replace it.

Dependencies and what it unlocks

Unlocks profiles, social relationships, groups, provider connections, and protected rooms.

Database / RLS / storage

Use additive profile completion/identity metadata. Protect private fields with RLS; never expose OAuth tokens or cookies.

Realtime / security

Realtime begins only after valid identity; redirect state cannot bypass authentication.

UI / UX expected result

Use current navigation and component vocabulary. Every async action has loading, disabled, success, error, empty, retry, accessible, and mobile states. Users see only data/features allowed by identity, membership, lifecycle, and permission.

Risks / regression prevention

- Add the smallest compatible seam and keep the current happy path available.
- Test direct server/database calls as well as UI.
- Use feature flags, additive migrations, and evidence gates; never use hidden UI as security.

Testing and verification

New/existing Google account, incomplete profile, logout/login, invalid OAuth state, redirect, SSR refresh, direct unauthenticated routes. Label every result separately as source/static, local runtime, multi-user/browser, database/security, or production.

Rollback / recovery

Feature flag or disable new onboarding without deleting users or auth records.

Definition of Done

Google and legacy-account flows work in supported environments, no duplicate users exist, and SSR/client identity agrees.

03

Database, RLS, Storage and Canonical Documentation

P0

PARTIAL

Action purpose / why it exists

Secure the data model before adding new relationships and content.

Current implementation and status

Migrations 002-004 and local 005 represent current security/lifecycle work; base schema and POLICIES.md contain historical definitions that must be clearly layered.

What must be fixed or built

Verify remote/local state, add only additive migrations, document effective policies, add constraints/indexes/immutable fields, restrict SECURITY DEFINER/search_path/grants, and scope storage by actual object paths.

Existing code/features to preserve

Preserve existing tables, lifecycle objects, room_messages compatibility, and working RLS. Never DROP TABLE, TRUNCATE, bulk-delete, replay 002/003, or create public bypasses.

Dependencies and what it unlocks

Unlocks social, rooms, groups, Music, and AI safely.

Database / RLS / storage

Every new table needs FKs, unique/status constraints, indexes, ownership/member RLS, and rollback notes. Storage and cleanup remain lifecycle-aware.

Realtime / security

RLS is authoritative; realtime publication/channel authorization must match it.

UI / UX expected result

Use current navigation and component vocabulary. Every async action has loading, disabled, success, error, empty, retry, accessible, and mobile states. Users see only data/features allowed by identity, membership, lifecycle, and permission.

Risks / regression prevention

- Add the smallest compatible seam and keep the current happy path available.
- Test direct server/database calls as well as UI.
- Use feature flags, additive migrations, and evidence gates; never use hidden UI as security.

Testing and verification

Migration dry run, schema diff, anonymous probes, full member-state matrix, storage path tests, direct server-action bypass tests. Label every result separately as source/static, local runtime, multi-user/browser, database/security, or production.

Rollback / recovery

Forward-compatible migrations and reversible policy/function changes.

Definition of Done

Remote objects match intended policy state and private-data probes deny unauthorized access.

04

Profiles, Privacy, Follows, Friends and Posts

P1

MISSING

Action purpose / why it exists

Create the identity graph used by profiles, member actions, Personal Chat, and General Chat.

Current implementation and status

Profiles exist, but complete setup, lowercase usernames, public/private visibility, posts, following, followers, friends, age/privacy rules, and account switching are incomplete.

What must be fixed or built

Add validated profile setup/editing, username uniqueness, derived age rules, public/private posts and lists, directional follow states, normalized friendship, privacy-aware profile actions, and safe friend-to-mutual-follow conversion.

Existing code/features to preserve

Reuse profiles/auth patterns. Keep following separate from friendship; do not expose friend counts, DOB, gender, activity, study history, or personal files by default.

Dependencies and what it unlocks

Unlocks social discovery, General Chat, Personal Chat, and profile actions.

Database / RLS / storage

Relationship tables use status checks, unique keys, recipient-only request decisions, post visibility RLS, and transactional mutual follows.

Realtime / security

Requests and relationship changes are recipient/owner-scoped realtime events.

UI / UX expected result

Use current navigation and component vocabulary. Every async action has loading, disabled, success, error, empty, retry, accessible, and mobile states. Users see only data/features allowed by identity, membership, lifecycle, and permission.

Risks / regression prevention

- Add the smallest compatible seam and keep the current happy path available.
- Test direct server/database calls as well as UI.
- Use feature flags, additive migrations, and evidence gates; never use hidden UI as security.

Testing and verification

Public/private visibility, requests, duplicates, unfollow/remove follower, friend states, privacy switching, age handling, direct RLS. Label every result separately as source/static, local runtime, multi-user/browser, database/security, or production.

Rollback / recovery

Additive tables/columns; preserve profiles/posts while disabling a rollout.

Definition of Done

Privacy and relationship matrix passes both UI and direct database tests.

05

Rooms, Membership, Roles, Invitations and Control

P1

PARTIAL

Action purpose / why it exists

Extend existing rooms without changing their established identity or access semantics.

Current implementation and status

createRoom/processRoomJoin support identifiers, public/private rooms, membership, usernames, expiration, and current settings.

What must be fixed or built

Add invite links/codes, leave/rejoin, kick/ban/block, rules/welcome, temporary guests, Control Center, granular permissions, ownership transfer, member 3-dot actions, room-specific profiles, Ghost Mode, locks, announcements, and feature enable/disable controls.

Existing code/features to preserve

Preserve room routes/actions, IDs/usernames, owner/admin/member model, presence, chat, Watch, Files, Study, notifications, and lifecycle.

Dependencies and what it unlocks

Unlocks safe collaboration and conversion.

Database / RLS / storage

Only add permission/settings records when existing columns are insufficient. RLS requires current approved membership, active lifecycle, and owner/admin grants.

Realtime / security

Removed/banned/ghost users cannot bypass realtime authorization; presence identity is scoped and auditable.

UI / UX expected result

Use current navigation and component vocabulary. Every async action has loading, disabled, success, error, empty, retry, accessible, and mobile states. Users see only data/features allowed by identity, membership, lifecycle, and permission.

Risks / regression prevention

- Add the smallest compatible seam and keep the current happy path available.
- Test direct server/database calls as well as UI.
- Use feature flags, additive migrations, and evidence gates; never use hidden UI as security.

Testing and verification

Creation/join, private approval, leave/rejoin, role boundaries, guests, Ghost Mode, disabled features, direct API bypass. Label every result separately as source/static, local runtime, multi-user/browser, database/security, or production.

Rollback / recovery

Flags/settings can be rolled back without deleting room content.

Definition of Done

All room controls agree across UI, server actions, RLS, storage, and realtime.

06

Lifecycle, Expiry, Recovery, Guests, Locks, Scheduling and Cleanup

P0

PARTIAL

Action purpose / why it exists

Define authoritative room state before collaboration depends on it.

Current implementation and status

room-lifecycle.ts, recovery actions, protected cleanup, cron contract, migration 004, reopened_until/permanent objects exist; runtime evidence and final semantics remain gates.

What must be fixed or built

Define active/expired/recovery-window/reopened/permanent/locked/scheduled/destroyed states; enforce 24-hour recovery window, exact 7-day reopen, permanent conversion without fake expiry, guests, schedules, room events, and idempotent cleanup.

Existing code/features to preserve

Reuse lifecycle helpers, recovery actions, cleanup route, cron, and existing fields where safe. Preserve recoverable data only; destroyed data never returns.

Dependencies and what it unlocks

Unlocks realtime, collaboration, conversion, exports, and production cleanup.

Database / RLS / storage

RPC/RLS enforce transitions; requests immutable except pending -> approved/rejected; storage/media cleanup is scoped and authorized.

Realtime / security

Expiry, recovery, permanent, guest, lock, and schedule events update every client and cut access promptly.

UI / UX expected result

Use current navigation and component vocabulary. Every async action has loading, disabled, success, error, empty, retry, accessible, and mobile states. Users see only data/features allowed by identity, membership, lifecycle, and permission.

Risks / regression prevention

- Add the smallest compatible seam and keep the current happy path available.
- Test direct server/database calls as well as UI.
- Use feature flags, additive migrations, and evidence gates; never use hidden UI as security.

Testing and verification

Forced expiry, window, review authorization, seven-day deadline, former-member denial, permanent state, guest expiry, cleanup repeatability. Label every result separately as source/static, local runtime, multi-user/browser, database/security, or production.

Rollback / recovery

Forward-compatible state machine and approved backups; never reset production.

Definition of Done

Database, server, UI, cleanup, and cross-client lifecycle behavior agree.

07

Shared Realtime Foundation

P1

PARTIAL

Action purpose / why it exists

Give room features one authorized synchronization path.

Current implementation and status

RoomRealtimeProvider/useRoomSync is shared by RoomChat, MediaStage, Files, Study, Members, and presence; duplicate room chat channels were removed.

What must be fixed or built

Define typed events for chat, Watch, seeks, files/PDFs, notes, study, Music, polls, notifications, activity, settings, presence, and lifecycle. Add deduplication, reconnect recovery, late-join snapshots, cleanup, and authorization.

Existing code/features to preserve

Preserve the shared provider and prohibit component-level duplicate room channels or presence providers.

Dependencies and what it unlocks

Unlocks multi-user collaboration and group workspace.

Database / RLS / storage

Persist important history where required; channel/table authorization matches membership/feature permission.

Realtime / security

Never put private keys, plaintext, provider tokens, or private account data in events.

UI / UX expected result

Use current navigation and component vocabulary. Every async action has loading, disabled, success, error, empty, retry, accessible, and mobile states. Users see only data/features allowed by identity, membership, lifecycle, and permission.

Risks / regression prevention

- Add the smallest compatible seam and keep the current happy path available.
- Test direct server/database calls as well as UI.
- Use feature flags, additive migrations, and evidence gates; never use hidden UI as security.

Testing and verification

Two independent browser profiles for delivery, ordering, duplicates, reconnect, late join, removal, expiry, and feature flags. Label every result separately as source/static, local runtime, multi-user/browser, database/security, or production.

Rollback / recovery

Revert new handlers while preserving the provider and database state.

Definition of Done

One authoritative room path passes multi-user and authorization tests.

08

Room Messaging and Notifications

P1

PARTIAL

Action purpose / why it exists

Make room chat a reliable privacy-safe collaboration surface.

Current implementation and status

RoomChat supports current chat, Delete for Everyone/Me, realtime, and separate E2EE private chat. Rich actions and complete notification state remain.

What must be fixed or built

Add edit/delete own messages, replies/threads, reactions, GIFs, mentions, pins, announcements, shared PDF notifications, member 3-dot actions, timer owner notifications, Notification Center, DND/mute, unread/read state, and history.

Existing code/features to preserve

Preserve RoomChat, chat actions, Delete for Everyone server/RLS, PrivateChat, allSettled decryption, and shared provider.

Dependencies and what it unlocks

Unlocks rich room communication, activity, and conversation history.

Database / RLS / storage

Message ownership/scope, update/delete RLS, immutable sender/room triggers, attachment references, mention scoping, and notification recipient RLS.

Realtime / security

Persist/realtime notifications without exposing private room events.

UI / UX expected result

Use current navigation and component vocabulary. Every async action has loading, disabled, success, error, empty, retry, accessible, and mobile states. Users see only data/features allowed by identity, membership, lifecycle, and permission.

Risks / regression prevention

- Add the smallest compatible seam and keep the current happy path available.
- Test direct server/database calls as well as UI.
- Use feature flags, additive migrations, and evidence gates; never use hidden UI as security.

Testing and verification

Two-user edit/delete/reply/reaction/mention/notification/unread tests and unauthorized direct calls. Label every result separately as source/static, local runtime, multi-user/browser, database/security, or production.

Rollback / recovery

Additive message metadata and feature flags; never rewrite ciphertext.

Definition of Done

Authorized actions work, unauthorized actions fail, and notifications are scoped/persistent where promised.

09

Adaptive Room Workspace

P1

MISSING

Action purpose / why it exists

Make room layout responsive to the active feature.

Current implementation and status

Room page has chat, stage, Files, Study, Members, and Focus Lock; final layout/preset/theme/control-center behavior is incomplete.

What must be fixed or built

Implement centered resizable chat, feature-open right chat, presets, dashboard, themes, feature visibility, saved panel sizes, mobile rules, and no empty feature gaps.

Existing code/features to preserve

Reuse room page, MediaStage, RoomChat, FilesTab, StudyStage, focus-lock, and provider; avoid unnecessary remounts.

Dependencies and what it unlocks

Unlocks comfortable Watch, Study, Files, Music, and group workspace.

Database / RLS / storage

Visible controls are not security: server/RLS/realtime must also deny disabled features.

Realtime / security

Shared layout events are authorized; private panel sizes remain user-scoped.

UI / UX expected result

Use current navigation and component vocabulary. Every async action has loading, disabled, success, error, empty, retry, accessible, and mobile states. Users see only data/features allowed by identity, membership, lifecycle, and permission.

Risks / regression prevention

- Add the smallest compatible seam and keep the current happy path available.
- Test direct server/database calls as well as UI.
- Use feature flags, additive migrations, and evidence gates; never use hidden UI as security.

Testing and verification

Desktop/mobile resize, feature open/close, disabled features, navigation, refresh, focus lock, subscription count. Label every result separately as source/static, local runtime, multi-user/browser, database/security, or production.

Rollback / recovery

Keep prior layout preset as fallback; revert preferences without data deletion.

Definition of Done

Layout, permissions, responsiveness, and feature mounting behave consistently.

10**Files, PDFs, Media and Storage Lifecycle**

P1

PARTIAL

Action purpose / why it exists

Make shared content safe and lifecycle-aware before Watch and collaboration expand.

Current implementation and status

Temporary media, storage helpers, FilesTab, and casting inputs exist; shared PDFs, permanent files, temporary links, metadata, lyrics, and robust upload workflows remain.

What must be fixed or built

Add shared PDFs with room-wide visibility/notification, permanent files, temporary signed links, multi-file progress/retry/cancel, metadata/artwork/lyrics/cache/attribution/fallback, subtitles/tracks, and safe upload retention.

Existing code/features to preserve

Preserve storage abstraction, FilesTab, MediaStage, room paths, temporary_media policies, and cleanup. Do not hard-code R2.

Dependencies and what it unlocks

Unlocks Watch, documents, clips, Music uploads, and exports.

Database / RLS / storage

Actual object paths, membership/lifecycle, type/size validation, bounded URLs, abandoned-upload cleanup, and no private downloads.

Realtime / security

Publish metadata/status, never private content or provider tokens.

UI / UX expected result

Use current navigation and component vocabulary. Every async action has loading, disabled, success, error, empty, retry, accessible, and mobile states. Users see only data/features allowed by identity, membership, lifecycle, and permission.

Risks / regression prevention

- Add the smallest compatible seam and keep the current happy path available.
- Test direct server/database calls as well as UI.
- Use feature flags, additive migrations, and evidence gates; never use hidden UI as security.

Testing and verification

Upload/read/delete/expiry, PDF visibility, signed link expiry, metadata fallback, storage bypass, cleanup. Label every result separately as source/static, local runtime, multi-user/browser, database/security, or production.

Rollback / recovery

Additive metadata/references; retain objects until verified replacement.

Definition of Done

Content is uploaded, shared, accessed, notified, expired, and cleaned safely.

11

Watch and Shared Media Reliability

P1

PARTIAL

Action purpose / why it exists

Complete synchronized Watch on secure media/realtime foundations.

Current implementation and status

React Player v3 remediation and local seek broadcasting exist; independent broad multi-user evidence remains.

What must be fixed or built

Verify src/currentTime usage, play/pause/seek suppression, throttling, drift correction, speed, subtitles/audio tracks, external/uploaded media, controls, late join, reconnect, shared queue, Now Playing, clips/highlights, and metadata.

Existing code/features to preserve

Preserve MediaStage, shared provider, Files, and permissions. Never reintroduce url, getInternalPlayer, config.file, or as any.

Dependencies and what it unlocks

Unlocks Watch parties, clips, metadata, and Lume Watch later.

Database / RLS / storage

Media state has active-room/member/control checks; sources use storage lifecycle and signed URLs.

Realtime / security

Remote state applies idempotently with no recursive rebroadcast.

UI / UX expected result

Use current navigation and component vocabulary. Every async action has loading, disabled, success, error, empty, retry, accessible, and mobile states. Users see only data/features allowed by identity, membership, lifecycle, and permission.

Risks / regression prevention

- Add the smallest compatible seam and keep the current happy path available.
- Test direct server/database calls as well as UI.
- Use feature flags, additive migrations, and evidence gates; never use hidden UI as security.

Testing and verification

Two independent profiles: play/pause/resume, both-direction seek, repeated seeks, late join, reconnect, subtitles, unsupported media, permissions. Label every result separately as source/static, local runtime, multi-user/browser, database/security, or production.

Rollback / recovery

Retain prior player path only as verified fallback; revert sync without changing data.

Definition of Done

Two users remain synchronized across controls, seeks, reconnect, media types, and permissions.

12

Study and Non-AI Collaboration

P2

PARTIAL

Action purpose / why it exists

Build normal human collaboration before AI summarizes or acts.

Current implementation and status

StudyStage, timer, Focus Lock, and study_sessions exist; notes, whiteboard, voice, screen share, raise hand, events, tasks, transcription, history/music, and lecture media need expansion.

What must be fixed or built

Add live notes, collaborative docs, whiteboard cursors, voice with owner mute, screen sharing, raise hand, polls, countdowns, manual tasks/action items, room events, study history/music, YouTube/lecture video, consented recording/transcription, and searchable history.

Existing code/features to preserve

Preserve StudyStage, timers, Focus Lock, room roles, media/files permissions, and cleanup. Do not let AI replace normal records.

Dependencies and what it unlocks

Unlocks AI inputs, minutes, summaries, tasks, and Room Memory.

Database / RLS / storage

Versioned documents, consent, retention/deletion, participant controls, and room/member RLS.

Realtime / security

Typed snapshots/events handle conflicts, reconnect, mute, cursors, and consented recording state.

UI / UX expected result

Use current navigation and component vocabulary. Every async action has loading, disabled, success, error, empty, retry, accessible, and mobile states. Users see only data/features allowed by identity, membership, lifecycle, and permission.

Risks / regression prevention

- Add the smallest compatible seam and keep the current happy path available.
- Test direct server/database calls as well as UI.
- Use feature flags, additive migrations, and evidence gates; never use hidden UI as security.

Testing and verification

Two-user edits, timer notification, voice mute, screen consent, raise hand, polls, tasks, transcription access, reconnect. Label every result separately as source/static, local runtime, multi-user/browser, database/security, or production.

Rollback / recovery

Gate advanced collaboration individually; preserve existing Study path.

Definition of Done

Every non-AI collaboration surface has clear owner, permission, persistence, retention, and evidence.

13

Activity, Discovery, Events and Management

P2

MISSING

Action purpose / why it exists

Make rooms useful to find, understand, and manage without reducing privacy.

Current implementation and status

Explore/search foundations exist; history, Pulse, bookmarks, templates, dashboards, trust, export, and smart organization are incomplete.

What must be fixed or built

Add Room Search and filters, Activity/Room History, Room Pulse/Live Activity Bar, Dashboard/Home, Bookmarks/Saved Content, templates, Smart Auto-Organization, Reputation/Trust, Cross-Room Dashboard, Backup/Export, and event management.

Existing code/features to preserve

Reuse Explore, metadata, history, storage refs, notifications, and shared realtime. Keep one owner for each feature; later actions integrate, not duplicate.

Dependencies and what it unlocks

Unlocks management, templates, conversion, and safe AI inputs.

Database / RLS / storage

Private results are membership-scoped; exports are owner/admin, signed, rate-limited, audited, and exclude keys.

Realtime / security

Activity is privacy-minimized and realtime only for eligible members.

UI / UX expected result

Use current navigation and component vocabulary. Every async action has loading, disabled, success, error, empty, retry, accessible, and mobile states. Users see only data/features allowed by identity, membership, lifecycle, and permission.

Risks / regression prevention

- Add the smallest compatible seam and keep the current happy path available.
- Test direct server/database calls as well as UI.
- Use feature flags, additive migrations, and evidence gates; never use hidden UI as security.

Testing and verification

Search privacy, history accuracy, export, bookmarks, trust boundaries, duplicate events, and cross-room isolation. Label every result separately as source/static, local runtime, multi-user/browser, database/security, or production.

Rollback / recovery

Disable new indexes/features without deleting records; preserve Explore.

Definition of Done

Discovery and management are useful, private, and auditable.

14

Room-to-Group Conversion

P1

PARTIAL

Action purpose / why it exists

Create one safe bridge from rooms to persistent Personal groups.

Current implementation and status

Current owner-only conversion accepts a group username and invokes a permanent-room RPC, conflicting with final group semantics.

What must be fixed or built

Use owner or explicit admin permission, eligibility checks, conversion lock, one persistent group, exact room name, no group username/link, originating-room reference, approved-member/content migration, and idempotence.

Existing code/features to preserve

Preserve room records, approved member semantics, eligible content/storage refs, and lifecycle history. Do not destroy room or resurrect deleted data.

Dependencies and what it unlocks

Unlocks Personal Groups and workspace.

Database / RLS / storage

Group/member/conversion records use RLS, unique constraints, all-or-nothing transaction, ownership, and audit.

Realtime / security

Publish one conversion event; notify legitimate migrated users only.

UI / UX expected result

Use current navigation and component vocabulary. Every async action has loading, disabled, success, error, empty, retry, accessible, and mobile states. Users see only data/features allowed by identity, membership, lifecycle, and permission.

Risks / regression prevention

- Add the smallest compatible seam and keep the current happy path available.
- Test direct server/database calls as well as UI.
- Use feature flags, additive migrations, and evidence gates; never use hidden UI as security.

Testing and verification

Double-click race, member inclusion/exclusion, username removal, content references, post-conversion RLS, rollback. Label every result separately as source/static, local runtime, multi-user/browser, database/security, or production.

Rollback / recovery

Stage references and conversion record; never delete source room in first conversion.

Definition of Done

Exactly one group exists with approved members/content, no public group username, and safe retry.

15

Personal, General and Group Messaging

P1

MISSING

Action purpose / why it exists

Build social messaging on identity, groups, and E2EE foundations.

Current implementation and status

messages page launches basic friend E2EE chat; Personal/General/Groups, requests, group messages, read state, and reclassification are missing.

What must be fixed or built

Add Personal Groups/All, General contacts/requests, friend/follow access, groups/members/roles, group messages, unread state, conversation types, and history-preserving General-to-Personal transition.

Existing code/features to preserve

Reuse usePrivateChat, PrivateChat, public-key helpers, allSettled UI, auth, and realtime patterns. Do not mix room messages or keys.

Dependencies and what it unlocks

Unlocks social communication and group Music.

Database / RLS / storage

Personal requires accepted friendship, General follow/accepted request, Groups membership, Rooms room membership.

Realtime / security

Authorized personal/group channels deduplicate and revoke access on block/leave/removal.

UI / UX expected result

Use current navigation and component vocabulary. Every async action has loading, disabled, success, error, empty, retry, accessible, and mobile states. Users see only data/features allowed by identity, membership, lifecycle, and permission.

Risks / regression prevention

- Add the smallest compatible seam and keep the current happy path available.
- Test direct server/database calls as well as UI.
- Use feature flags, additive migrations, and evidence gates; never use hidden UI as security.

Testing and verification

Four-account relationship/E2EE matrix, request gating, reclassification, group membership, unauthorized reads/writes, refresh/realtime. Label every result separately as source/static, local runtime, multi-user/browser, database/security, or production.

Rollback / recovery

Roll out routes without removing current private-chat path; relabel metadata, not ciphertext.

Definition of Done

Every conversation has one type, correct access, safe decryption failure, and no duplicate entries.

16

Personal and Shared Music

P2

MISSING

Action purpose / why it exists

Add Music after private storage, groups, and shared state exist.

Current implementation and status

No complete personal/provider Music system exists.

What must be fixed or built

Build personal player, multi-upload with 48-hour expiry, metadata/artwork/lyrics, queue, secure adapters for Spotify/Apple Music/TIDAL/Qobuz/YouTube Music, provider selection/search/playlists, official limitations, and room/group Music.

Existing code/features to preserve

Reuse storage, Files lifecycle, media metadata, group/room permissions, and layout. Never hard-code R2, scrape providers, or expose tokens.

Dependencies and what it unlocks

Unlocks Lume Music later.

Database / RLS / storage

Tokens remain server-side/revocable; uploads are private and expire with metadata cleanup; provider rules limit shared playback.

Realtime / security

Personal playback stays personal; shared references/state use authorized provider.

UI / UX expected result

Use current navigation and component vocabulary. Every async action has loading, disabled, success, error, empty, retry, accessible, and mobile states. Users see only data/features allowed by identity, membership, lifecycle, and permission.

Risks / regression prevention

- Add the smallest compatible seam and keep the current happy path available.
- Test direct server/database calls as well as UI.
- Use feature flags, additive migrations, and evidence gates; never use hidden UI as security.

Testing and verification

Formats, uploads, retry/cancel, expiry, OAuth, playlists, limits, shared controls, disabled Music, token leakage, RLS. Label every result separately as source/static, local runtime, multi-user/browser, database/security, or production.

Rollback / recovery

Provider adapters are replaceable; disconnect a provider without deleting uploads.

Definition of Done

Music is secure, honest about limits, synchronized where allowed, and lifecycle-safe.

17**AI and Intelligence Last**

P2

PLANNED

Action purpose / why it exists

Add intelligence only after normal records and permissions are trustworthy.

Current implementation and status

Lume Study/Watch/Music, room assistant, meeting assistant, summaries, minutes, tasks, and Room Memory are product requirements, not completed features.

What must be fixed or built

Build permission-aware AI assistant, live meeting assistant, AI task drafts, summaries, minutes, and Room Memory. Require consent, source references, data minimization, user approval, retention/deletion, model abstraction, and safe logs.

Existing code/features to preserve

Reuse verified chat/history, notes, files, Watch, Music metadata, calls/transcription, manual tasks, activity, and search. AI never becomes a new source of truth or access path.

Dependencies and what it unlocks

Unlocks intelligent product value without weakening privacy.

Database / RLS / storage

Recheck current membership at retrieval time; redact secrets; defend against prompt injection; support deletion and rate limits.

Realtime / security

AI actions are authorized server calls; results go only to eligible users.

UI / UX expected result

Use current navigation and component vocabulary. Every async action has loading, disabled, success, error, empty, retry, accessible, and mobile states. Users see only data/features allowed by identity, membership, lifecycle, and permission.

Risks / regression prevention

- Add the smallest compatible seam and keep the current happy path available.
- Test direct server/database calls as well as UI.
- Use feature flags, additive migrations, and evidence gates; never use hidden UI as security.

Testing and verification

Consent, sources, wrong answers, task approval, deletion, expiry, membership change, leakage, abuse, rate limits. Label every result separately as source/static, local runtime, multi-user/browser, database/security, or production.

Rollback / recovery

Disable AI without changing normal records; preserve manual systems.

Definition of Done

Every AI feature is permission-aware, source-linked, user-controllable, tested for leakage, and backed by verified non-AI systems.

18

Hardening, Operations and Production Gate

P0

UNVERIFIED

Action purpose / why it exists

Turn the product into a release candidate only after feature stages have evidence.

Current implementation and status

TypeScript/build/lint and several security/deployment checks have passed historically; runtime breadth, Cron, mobile, performance, and some product flows remain.

What must be fixed or built

Run full testing ladder, accessibility/mobile, performance, monitoring, safe logs, backups/restore, migration/schema verification, Cron evidence, runbooks, deployment smoke, and recheck Actions 1-17.

Existing code/features to preserve

Preserve verified architecture; fix only release-blocking or remediation-caused defects.

Dependencies and what it unlocks

Unlocks a defensible production decision.

Database / RLS / storage

Production gate requires remote alignment, anonymous denial, RLS matrix, storage, lifecycle, notifications, conversion, E2EE, SSR, and no critical/high issue.

Realtime / security

Monitor safe metrics: channel counts, latency, cleanup, auth/storage failures; never log secrets/plaintext/ciphertext.

UI / UX expected result

Use current navigation and component vocabulary. Every async action has loading, disabled, success, error, empty, retry, accessible, and mobile states. Users see only data/features allowed by identity, membership, lifecycle, and permission.

Risks / regression prevention

- Add the smallest compatible seam and keep the current happy path available.
- Test direct server/database calls as well as UI.
- Use feature flags, additive migrations, and evidence gates; never use hidden UI as security.

Testing and verification

Typecheck, build, lint, automated tests, security probes, four-account browser tests, mobile/accessibility, reconnect, Cron logs, deployment verification. Label every result separately as source/static, local runtime, multi-user/browser, database/security, or production.

Rollback / recovery

Feature flags, Vercel rollback, additive migrations, backups; never reset production.

Definition of Done

Every release gate has evidence and readiness can be declared without inference.

Security, Database and Realtime Rules

Area	Rule	Evidence gate
Authentication	SSR session refresh, safe OAuth state, protected routes, no secrets.	Login/logout/refresh/redirect browser and server tests.
RLS and storage	No anonymous/unrelated private data; room/member/lifecycle/storage paths are scoped.	Direct anonymous/authenticated matrix and storage bypass tests.
Messages/E2EE	Room messages use room authorization; private keys stay device-side; per-message decrypt failure is safe.	Ownership/delete/update and mixed-key runtime tests.
Realtime	One room provider with authorized events, deduplication, reconnect, late join, and removal handling.	Independent two-user browser and reconnect evidence.
Lifecycle	Expiry/recovery/permanent/guest/lock/schedule are database/server-enforced.	Forced transitions, RPC/RLS, cleanup logs, cross-client tests.
Logging	No keys, plaintext, ciphertext, tokens, cookies, or sensitive crypto errors.	Source search, browser console, server log review.

Verification and Evidence Gates

Gate	What it proves	Not enough by itself
G1 Static	Types/build/lint/routes/migrations/searches are coherent.	Passing build alone.
G2 Local runtime	One configured environment works.	Source existence.
G3 Multi-user	Independent identities exchange intended state.	Two tabs sharing cookies.
G4 Database/security	RLS/storage/constraints/functions block bypasses.	UI hiding or server action checks alone.
G5 Production	Deployed commit, remote schema, Cron/logs, and routes behave.	A successful deployment command.

Master Feature Register

All 54 newly requested features are integrated into Actions 05-17. The register below assigns one owning action; later references are reuse/integration, not duplicate implementations.

Domain	Feature	Status	Owner
Room / Collaboration	Edit/delete own messages	Partial	08
Room / Collaboration	Shared PDFs	Partial	10/12
Room / Collaboration	Invite link + unique code	Missing	05/09
Room / Collaboration	Live shared notes	Missing	12
Room / Collaboration	Timer-created owner notification	Missing	12
Room / Collaboration	Member 3-dot actions	Missing	08
Room / Collaboration	Ghost Mode	Missing	05/09
Room / Collaboration	Voice chat + owner mute	Partial	12
Room / Collaboration	Room Control Center	Missing	05/09
Room / Collaboration	Room Roles & Permissions	Missing	05/09
Room / Collaboration	Room Announcements	Missing	08
Room / Collaboration	Pinned Messages	Missing	08
Room / Collaboration	Message Replies / Threads	Missing	08
Room / Collaboration	Mentions	Missing	08
Room / Collaboration	Notification Center	Missing	08
Room / Collaboration	Room Search	Partial	13
Room / Collaboration	Activity / Room History	Missing	13
Room / Collaboration	Custom Room Layout Presets	Missing	05/09
Room / Collaboration	Owner-controlled room UI/layout	Missing	05/09
Room / Collaboration	Room Themes / Appearance	Missing	05/09
Room / Collaboration	Polls & Voting	Missing	12
Room / Collaboration	Shared Queue	Missing	11
Room / Collaboration	Now Playing / Shared State Bar	Missing	11
Room / Collaboration	User Presence States	Missing	05/09
Room / Collaboration	Leave / Kick / Ban / Block	Missing	05/09
Room / Collaboration	Screen Sharing	Missing	12
Room / Collaboration	Raise Hand	Missing	12
Room / Collaboration	Room Rules & Welcome Message	Missing	05/09
Room / Collaboration	Temporary Guest Access	Missing	05/09
Room / Collaboration	Room Lock	Missing	05/09
Room / Collaboration	Scheduled Rooms	Missing	12
Room / Collaboration	Room Events	Missing	12
Room / Collaboration	Shared Whiteboard Cursor Presence	Missing	10/12
Room / Collaboration	Collaborative Document Editing	Missing	10/12

Domain	Feature	Status	Owner
Room / Collaboration	Room Bookmarks / Saved Content	Missing	05/09
Room / Collaboration	Room-Specific User Profiles	Missing	05/09
Room / Collaboration	Smart Room Search & Filters	Missing	13
Room / Collaboration	Room Backup / Export	Missing	13
Room / Collaboration	Shared Countdown / Event Timer	Missing	10/12
Room / Collaboration	Live Activity Bar	Missing	13
Room / Collaboration	Room Dashboard / Home	Missing	13
Room / Collaboration	Room Clips / Highlights	Missing	11
Room / Collaboration	Smart Auto-Organization	Missing	13
Room / Collaboration	Room Templates	Missing	13
Room / Collaboration	Guest / Temporary Identity Mode	Missing	05/09
Room / Collaboration	Room Reputation / Trust Levels	Missing	13
Room / Collaboration	Smart Notification Controls / Do Not Disturb	Missing	08
Room / Collaboration	Cross-Room Personal Dashboard	Missing	13
AI & Intelligence	AI Room Assistant	Planned	17
Room / Collaboration	Live AI Meeting Assistant	Missing	05/09
AI & Intelligence	AI-assisted Action Items / Tasks	Planned	17
AI & Intelligence	Automated Room Summary	Planned	17
AI & Intelligence	Meeting Minutes / Meeting Summary	Planned	17
AI & Intelligence	Room Memory	Planned	17

Testing Ladder and Production Checklist

Testing ladder

Level 1: typecheck, build, lint, diff-check, static searches.

Level 2: unit tests for validators, state transitions, parsers, crypto wrappers.

Level 3: direct Supabase/RLS/storage tests with disposable identities.

Level 4: authenticated server-action/lifecycle integration.

Level 5: independent browser profiles for realtime, Watch, chat, presence, deletion, conversion.

Level 6: production smoke, Cron evidence, SSR refresh, mobile/accessibility, performance.

Release gates

Gate 1: secure foundation.

Gate 2: core rooms and lifecycle.

Gate 3: Watch.

Gate 4: non-AI collaboration.

Gate 5: social/groups/Music.

Gate 6: AI only after normal systems.

Gate 7: hardening and production.

Production Readiness Checklist

Area	Must be true before Production Ready = YES
Product	Auth, rooms, Watch, chat, Files, Study, social/groups, Music limits, and core UX work in supported environments.
Security	Anonymous/private probes, RLS matrix, storage, former-member denial, E2EE isolation, and realtime authorization pass.
Database	Remote migrations/docs align; lifecycle, recovery, cleanup, conversion, notifications, and constraints are verified.
Realtime	One provider, multi-user sync, presence, reconnect, deduplication, late join, membership-loss handling.
UX	Loading/error/empty/accessibility/mobile states work; no hidden-feature bypass or empty workspace gaps.
Operations	Cron evidence, monitoring, safe logs, backups, runbooks, deployment verification, rollback path.
AI	Notes, files, history, tasks, transcription, summaries, and permission boundaries are complete before AI.

Final readiness position

PRODUCTION READY = NO until evidence exists. The roadmap is corrected for implementation planning, but planning itself is not proof that the product is complete.